

Re7 Labs Security Assessment Report

August 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
 - Security Review Lead
 - Scope
 - Changelog
4. Overview
5. Severity Structure
 - Severity characteristics
 - Issue symbolic codes
6. Findings Summary
7. Weaknesses
 - Terms acceptance can redirect users to an external website
 - Unbounded wallet read RPC calls can exhaust backend resources
 - Standard redemption approval occurs before minimum-amount validation
 - Redemption can prompt approval for an unsupported payout token

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report presents the results of the security review of Re7 Labs' Midas application. The assessment was conducted from 20 to 31 August 2026 and covered the scoped application and supporting backend services.

Our review evaluated the application's architecture, wallet and transaction security, user-facing workflows, and backend resilience. During the assessment, we did not identify any Critical, High, or Medium-severity vulnerabilities. We identified two Low-severity findings and two Informational findings, primarily related to application flow validation and operational robustness.

The assessment confirmed that the application's core authorization and transaction security boundaries were appropriately designed within the reviewed scope. We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

- **Review Led by**

Jahyun Koo, Lead Security Researcher

- **Scope**

The analyzed resources are located on:

<https://github.com/solvedefi/re7-midas-dapp/tree/bbd16111e0ee8f75d2cd22f7cd0b7bf914d327b8>

The issues described in this report were fixed in the following commit:

<https://github.com/solvedefi/re7-midas-dapp/tree/2ab6b52b771f248568e839fee58d6982d5382f53>

- **Changelog**

■ 20 August 2026	Audit start
■ 31 August 2026	Initial report
■ 1 September 2026	Revision received
■ 3 September 2026	Final report

4. Overview

▪ Access Control

The application does not define privileged in-app roles. State-changing strategy operations are authorized only by the connected wallet, through ERC-20 approve and the corresponding Midas call. Terms acceptance is a product gate on the UI. It does not grant the backend or another party signing authority.

▪ Phishing Protection

We reviewed the paths that could move a user from the genuine host origin onto a hostile site: wallet connection, terms acceptance, post-acceptance navigation, and transaction construction. We verified that spend targets and calldata are built from bundled vault addresses and typed ABIs, not from user-supplied destinations. We confirmed that terms and transaction prompts are shown by the external wallet, so host cannot substitute a fake signing UI.

We also confirmed that return navigation after terms acceptance is intended to allow only same-origin paths. One implementation gap remains: a rejected external URL can still be followed after a successful terms flow.

▪ Transaction Integrity

We reviewed how invest and redeem requests are built, prompted, and confirmed. We verified that calldata is encoded from typed ABIs against bundled vault and token addresses, not user-supplied targets. We confirmed that writes are simulated before the wallet prompt, and that mined transactions are re-read on the application RPC and checked for chain, sender, target, and input.

Approval can still run before the application knows the following redemption will succeed. Below-minimum amounts and unsupported payout tokens may mine an unused vault allowance. Both are Informational.

▪ Web Vulnerabilities

We reviewed the browser-facing HTTP surface: route handling, search parameters, post-acceptance navigation, activity links, and mutating API endpoints. We verified that explorer URLs are built from a fixed HTTPS base rather than from untrusted schemes. We confirmed that production fund-moving routes were not missing origin protection.

- **Policy and Presentation**

We reviewed the stated approval policy against the invest and redeem sequence. We verified that the UI presents approvals as exact for the current action and as leaving no standing allowance. We confirmed that this holds when the following vault call succeeds, and that the spender is the configured vault.

5. Severity Structure

The vulnerability severity is calculated based on two components:

- 1. Impact of the vulnerability
- 2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical	Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.
High	Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

6. Findings Summary

Severity

Number of findings

Critical	0
High	0
Medium	0
Low	2
Informational	2

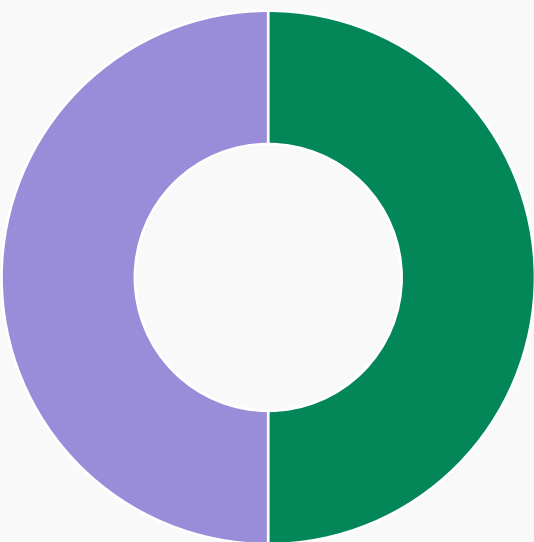
Total:

4



Low

Informational



Fixed

Acknowledged

7. Weaknesses

This section contains the list of discovered weaknesses.

RE72-1 | Terms acceptance can redirect users to an external website

Fixed 

Severity:

Low

Description:

The `/terms` route attempts to restrict the redirect parameter to same-origin paths. Absolute URLs are rejected by `safeRedirect`:

```
export function safeRedirect(value: unknown): string | undefined {
  if (typeof value !== "string" || !value.startsWith("/")) return undefined;
  const second = value[1];
  return second === "/" || second === "\\" ? undefined : value;
}
```

However, the route omits redirect from the validated object when validation fails:

```
export const termsRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: "/terms",
  component: TermsPage,
  validateSearch: (search: Record<string, unknown>): { redirect?: string } => {
    const redirect = safeRedirect(search.redirect);
    return redirect ? { redirect } : {};
  },
});
```

TanStack Router merges this object over the raw parent search. Because the rejected key is omitted rather than overwritten, the original absolute URL remains available to the component. Following successful terms acceptance, it is passed directly to the navigation sink:

```
const onAccept = () => {
  if (!address) return;
  accept.mutate(address, {
```

```
onSuccess: () => {  
  toast.success("Terms accepted", {  
    description: "You can now deposit in our strategies.",  
  });  
  if (redirect) navigate({ href: redirect });  
  else navigate({ to: "/" });  
},
```

An attacker can send a genuine Re7 URL such as **/terms?**

redirect=https%3A%2F%2Fattacker.example%2Fcontinue. If a connected user who has not accepted the current terms completes the normal acceptance flow, the application automatically navigates the tab to the external origin. This creates a product-mediated origin-confusion and phishing-delivery path.

Remediation:

- Explicitly overwrite rejected redirect values during search validation instead of omitting the key.
- Revalidate and parse the destination immediately before navigation, allowing only URLs whose resolved origin matches the application origin.
- Use internal-route navigation for accepted return destinations rather than arbitrary href navigation.

RE72-4 | Unbounded wallet read RPC calls can exhaust backend resources

Fixed ✓

Severity:

Low

Description:

apps/server/src/app.ts:createApp#L37-L43

apps/server/src/modules/portfolio/portfolio.service.ts:readPortfolio#L11-L38

apps/server/src/modules/activity/activity.service.ts:readActivity#L89-L125

The public **/api/portfolio/:address** and **/api/activity/:address** endpoints return a wallet's positions and settled deposit or redemption history. Both routes accept any valid EVM address and are installed without authentication or rate limiting. A portfolio request reads token balances across every deployment, while an activity request searches both vaults of every deployment through the configured explorers.

These reads should have bounded resource usage. Expensive upstream work should be rate limited, repeated requests for the same address should share an in-flight load, and queued work and cached entries should have hard size limits.

However, every portfolio request makes five uncached RPC calls. A cache miss can schedule 20 explorer calls when both explorer keys are configured: two topic queries for each deposit and redemption vault across five deployments. The code can delay these calls but does not limit the queue, so every request immediately reserves more future work and keeps its promises, timers, and request state in memory.

The activity cache does not join concurrent misses, meaning many simultaneous requests for the same address all schedule the full load. Its backing Map also has no capacity limit and does not remove expired entries. For example, 1,000 quick activity requests can reserve roughly 12,000 Etherscan slots. At four calls per second, that keeps the shared queue busy for about 50 minutes and may exhaust the backend's memory. Portfolio flooding can similarly consume the shared RPC quota and break other reads.

An attack can happen as follows:

1. An unauthenticated attacker sends many concurrent portfolio or activity requests using valid wallet addresses.
2. Portfolio requests create 5N RPC calls, while activity cache misses create up to 20N explorer calls and add them to the unbounded pacer queues.
3. Legitimate reads are delayed or fail after upstream quotas are exhausted. With a large enough activity backlog, retained work can exhaust memory and restart the backend process.

Remediation:

Add per-IP rate limits and global concurrency or queue limits to both endpoints, maintain cache and batch portfolio reads under a global RPC budget.

RE72-2 | Standard redemption approval occurs before minimum-amount validation

Acknowledged

Severity:

Informational

Description:

The Standard redemption flow accepts any positive amount without reading or enforcing the selected vault's **minAmount**.

```
const amounts = useMemo(() => {
  const clean = amount.endsWith(".") ? amount.slice(0, -1) : amount;
  if (clean === "" || Number(clean) <= 0 || approveDecimals === undefined) return undefined;
  try {
    return {
      contract: parseUnits(clean, 18),
      allowance: parseUnits(clean, approveDecimals),
    };
  } catch {
    return undefined;
  }
}, [amount, approveDecimals]);
```

When the current allowance is insufficient, the submission logic completes the approval step and returns. The redemption is constructed and simulated only during the subsequent action step.

```
const submit = () => {
  ...
  if (needsApproval) {
    approveMutation.mutate();
    return;
  }
  actionMutation.mutate();
};
```

As a result, Re7 may request and submit an ERC-20 approval even when the entered amount is below the vault's minimum redemption amount. The minimum is checked only during the subsequent redemption preflight, after the approval has already been mined. This leaves the user with an unnecessary transaction fee and a residual allowance for a redemption that cannot proceed.

Remediation:

- Add the authoritative minAmount getter to the redemption ABI, vault-state reader, and state schema.
- Validate the entered amount against the selected vault's current minimum before enabling approval, failing closed when the minimum is unavailable.

RE72-3 | Redemption can prompt approval for an unsupported payout token

Acknowledged

Severity:

Informational

Description:

The application reads supported payment tokens only from the deposit vault:

```
export async function readVaultState(  
  client: PublicClient,  
  vault: VaultAddresses,  
): Promise<VaultState> {  
  ...  
  client.readContract({  
    address: vault.depositVault,  
    abi: depositVaultAbi,  
    functionName: "getPaymentTokens",  
  }),  
}
```

The same list is also used to select the payout token in redemption mode:

```
const paymentTokenMeta = paymentTokens.find((token) => token.address === effectivePayToken);  
const vaultTokenMeta = vaultTokenAddress  
  ? { address: vaultTokenAddress, decimals: 18 }  
  : undefined;  
const payTokenMeta = mode === "invest" ? paymentTokenMeta : vaultTokenMeta;  
const receiveTokenMeta = mode === "invest" ? vaultTokenMeta : paymentTokenMeta;
```

When the deposit and redemption vaults support different tokens, the interface can therefore offer a payout token that the redemption vault does not accept. The approval is submitted before the redemption action is simulated:

```
const submit = () => {  
  ...  
  if (needsApproval) {  
    approveMutation.mutate();  
    return;  
  }  
}
```

```
actionMutation.mutate();  
};
```

If an unsupported payout token is selected, the approval step can complete before the redemption is validated. The subsequent redemption is rejected, leaving the approval unused.

Remediation:

- Read and maintain separate supported-token lists for the deposit and redemption vaults.
- Validate the selected payout token against the redemption vault's list before enabling approval.

hexens x  Re7 Labs

